

AMSS 2026 — Lab 1: Tooling Onboarding + Requirements with AI

Traian-Florin Șerbănuță

2026

Lab 1: Requirements with AI

Three phases, 100 minutes:

1. **Tooling onboarding** (~15 min) — activate your AI endpoint, confirm repo access.
2. **Requirements drill** (~60 min) — in pairs, drive AI to produce a requirements doc; critique it.
3. **Share-out** (~25 min) — compare failure modes across the room.

Deliverable: a requirements doc + a 5-line reflection, committed to the course lab repo.

Before You Start

- ▶ Continue.dev installed per `tooling/SETUP.md` — you did this *before* lab.
- ▶ You receive at lab start: your model-endpoint credentials, your **pair-id**, and the lab-repo URL.
- ▶ Find your partner. This is a pair lab.

Phase 1 — Onboarding (15 min)

1. Paste your three credentials (model, apiBase, apiKey) into `tooling/.continue/config.yaml`.
2. Smoke test: open any file, press `Ctrl+L`, ask *“summarize this file”*. A coherent answer means the endpoint is live.
3. Clone or pull the lab repo, create branch `lab01/<pair-id>`, and push a stub to confirm access.

Stuck? Endpoint fails → switch to the Gemini free-tier config (`tooling/SETUP.md`). Continue broken → work on your partner's laptop.

Phase 2 — The Drill (60 min)

Domain: a vending machine. Same for every pair.

You alternate two roles:

- ▶ **Architect** — drives the AI (writes the prompts).
- ▶ **Critic** — reads the AI's output and names what's wrong or missing.

Swap roles between the two rounds, so you both practice both.

Round 1 — A drives, B critiques (~25 min)

A sends this starting prompt (deliberately bare):

“Generate a requirements document for a vending machine. Cover functional, non-functional, and domain requirements.”

B reads the output and names failure modes (see the next slides). Log each one.

A re-prompts using a prompting move to fix what B found. **This re-prompt is required — iterate at least once.**

Round 2 — swap: B drives, A critiques (~20 min)

Now **B** drives and **A** critiques.

Take a fresh angle: push for measurable acceptance criteria on each requirement, or negative-prompt away the fabricated technology A's round surfaced.

Keep a running log of *what you asked the AI and why* — that's your F3 evidence and the source of reflection line 3.

Finalize + Commit (last 10 min)

- ▶ Keep the best requirements doc you drove the AI to — your tightest iteration, not the first draft.
- ▶ Write the 5-line reflection (template a few slides on).
- ▶ Commit and push to your pair branch.

Three Prompting Moves (from W2)

- ▶ **Scaffolded** — *“organized by use case; each use case is one user goal; list FR / NFR / domain per case.”*
- ▶ **Role priming** — *“act as a requirements engineer for vending hardware.”*
- ▶ **Negative prompting** — *“no technology choices; no invented payment methods or hardware.”*

Use at least one when you re-prompt.

Failure Modes to Catch (from W2)

Five named modes — what AI wrongly **includes** or mis-states:

- ▶ **Fabrication** — invented stakeholders, features, or regulations.
- ▶ **Over-specification** — dozens of requirements where a handful suffice.
- ▶ **Vague NFRs** — “fast”, “reliable” with no measurable threshold.
- ▶ **Conflated requirements** — one requirement bundling several concerns.
- ▶ **Fabricated technology** — implementation choices posing as requirements.

Plus the flip side — **omission** — what AI silently **leaves out**.

Critic's Card — Vending Machine

If the draft looks complete, stress these. Most are omissions:

- ▶ Exact change unavailable / making change
- ▶ Item sold out after selection
- ▶ Two buyers grab the last item at once
- ▶ Power loss mid-vend (paid, not dispensed)
- ▶ Refund / cancel before dispense
- ▶ Restock & cash audit (operator role)
- ▶ *“Shall be fast / reliable”* — vague NFR
- ▶ *“Use an SQL database / a specific coin mechanism”* — fabricated technology

A draft that omits these *is* failing — silently.

Deliverable

On branch `lab01/<pair-id>`, commit:

- ▶ `lab01/<pair-id>/requirements.md` — your best AI-driven requirements doc.
- ▶ `lab01/<pair-id>/reflection.md` — exactly 5 lines (next slide).
- ▶ `lab01/<pair-id>/log.md` — optional but recommended: your prompt/critique log.

The 5-Line Reflection

One line each:

1. Worst failure mode you saw (name it) + where it appeared.
2. A second failure mode + why it was easy or hard to catch.
3. What you changed in the re-prompt — and **why** (this is F3).
4. What improved (or didn't) after iterating.
5. One requirement the AI never got right — the residual risk.

How to Submit

```
git checkout lab01/<pair-id>      # the branch you created
mkdir -p lab01/<pair-id>
# write requirements.md and reflection.md inside lab01/<pair-id>
git add lab01/<pair-id>
git commit -m "Lab 1: <pair-id> vending machine requirement"
git push -u origin lab01/<pair-id>
```

Push fails? Paste both files into the shared doc / email the instructor, then fix git after class.

Grading — Pass / Redo

Pass needs both:

1. Both files committed by the deadline.
2. Your reflection names **at least 2 failure modes** correctly **and** line 3 gives a real reason for your re-prompt (not just what you typed).

A vacuous reflection (“AI was wrong, we fixed it”) is a redo, not a fail. We grade your critique and reasoning — not the AI’s output quality.

Phase 3 — Share-out (25 min)

- ▶ A few pairs present their worst failure mode + the move that fixed it.
- ▶ We tally, live, which failure modes hit the most pairs.
- ▶ Same domain, same starting prompt, ~50 pairs → a real failure-mode map.

The modes you tally are exactly what you'll critique every week.

Why This Matters

If requirements are fabricated, every downstream artifact inherits it — class diagrams (Lab 2), tests (W3), all of it.

Today you drilled **F1** (read & critique AI output) and **F3** (say *why* you directed the AI). Next: Lab 2 turns requirements into class diagrams.